

Project for 6th Semester of Bachelor of Information Technology

PLUMMET

A 2D Puzzle-Platformer for Android



Submitted by

Student	Class Roll No.
Mausham Neupane	360379
Mohammad Aarman	360381
Tapendra Shahi	360400

KIST College of Information Technology

Kamalpokhari, Kathmandu

Submitted to

Faculty of Science and Technology

Purbanchal University

August 2026

STUDENT DECLARATION

We hereby declare that the project report entitled “Plummet: A 2D Puzzle-Platformer for Android” is an original account of the software system developed and examined by us for the sixth semester of the Bachelor of Information Technology program. The report has been prepared from the current Unity project, its complete authored source code, scene and prefab configuration, project settings, and direct Play Mode observation. Material from the earlier Rock VS Mine report has been used only as an academic layout and structural reference.

The work has not been submitted elsewhere for an academic award. All external frameworks, packages, services, and documentation consulted during development or preparation of this report are acknowledged in the references.

Student	Registration No.	Signature
Mausham Neupane	058-3-2-05288-2022	_____
Mohammad Aarman	058-3-2-05289-2022	_____
Tapendra Shahi	058-3-2-05310-2022	_____

Date: _____

RECOMMENDATION

This is to certify that the project report entitled “Plummet: A 2D Puzzle-Platformer for Android” has been prepared by Mausham Neupane, Mohammad Aarman, and Tapendra Shahi under the academic requirements of the Bachelor of Information Technology program. The report presents the design, implementation, and verification of the submitted Unity project and is recommended for evaluation.

The technical account is organized around observable project evidence: the runtime architecture, gameplay state model, data-driven level-action system, platform integrations, content catalog, and current validation status.

Deepak Khadka

Coordinator, B.I.T.

Bishal Khatri

Project Supervisor

CERTIFICATE

This is to certify that the project entitled “Plummet: A 2D Puzzle-Platformer for Android” has been examined and accepted as a partial fulfillment of the requirements for the Bachelor of Information Technology degree of Purbanchal University.

The undersigned certify that the report describes the submitted project in a clear and professionally structured manner, subject to the regulations and final evaluation procedures of the University.

Pankaj Jaiswal
External Examiner

Bishal Khatri
Project Supervisor

Deepak Khadka
Coordinator, B.I.T.

ACKNOWLEDGEMENT

We express our sincere gratitude to KIST College of Information Technology and the Faculty of Science and Technology, Purbanchal University, for providing the academic setting in which this project was completed. We are especially thankful to our project supervisor, Mr. Bishal Khatri, and our program coordinator, Mr. Deepak Khadka, for their guidance, review, and encouragement.

We also acknowledge the developers and maintainers of Unity, VContainer, UniTask, PrimeTween, Firebase, Google Mobile Ads, Smartlook, and the other packages used by the project. Their tools enabled us to concentrate on gameplay design, reliable scene orchestration, responsive input, and mobile deployment.

Finally, we thank our classmates, friends, and family for testing, feedback, and continued support throughout the design and implementation of Plummet.

ABSTRACT

Plummet is a two-dimensional puzzle-platformer developed in Unity for Android. The player navigates compact pixel-art levels by moving and jumping through hazards, falling platforms, moving geometry, rule-changing triggers, and exit doors. Its central design idea is that a level may alter the assumptions under which the player acts: controls can invert, gravity and jump behavior can change, and control can transfer to a puppet avatar. These changes are implemented through a reusable, serializable action framework rather than a separate controller for every level.

The software uses a layered runtime architecture. VContainer establishes dependency injection scopes; a signal bus coordinates menu, player, and level events; additive scene loading separates bootstrap, menu, and gameplay responsibilities; and a player finite-state machine isolates idle, running, jumping, locked, and dead behavior. PlayerPrefs stores progress and audio settings. Firebase Analytics, Smartlook, Google Mobile Ads, and Google Play in-app updates provide platform capabilities behind service interfaces.

This report is grounded in a complete inspection of the authored scripts, scenes, prefabs, package manifest, project settings, level catalog, and a Play Mode pass. The project contains 115 authored C# files totaling 7,956 lines, three build scenes, 57 prefabs, and 21 cataloged levels. A Level 22 prefab is present but not included in the runtime catalog. The system provides a maintainable mobile puzzle-platformer foundation with clear opportunities for automated testing, catalog validation, and production service configuration.

TABLE OF CONTENTS

CHAPTER 1 INTRODUCTION	1
1.1 Background	1
1.2 Problem Statement	1
1.3 Objectives.....	1
1.4 Scope	2
CHAPTER 2 REQUIREMENTS ANALYSIS AND FEASIBILITY	3
2.1 Functional Requirements	3
2.2 Non-Functional Requirements	3
2.3 Feasibility.....	4
2.4 Constraints and Assumptions.....	4
CHAPTER 3 SYSTEM METHODOLOGY AND DESIGN	5
3.1 Development Method.....	5
3.2 Runtime Architecture	5
3.3 Composition and Scene Lifecycle.....	5
3.4 Player Finite-State Machine.....	6
3.5 Level Lifecycle	6
3.6 Data-Driven Level Actions	7
3.7 Data and Integration Design	8
CHAPTER 4 IMPLEMENTATION.....	9
4.1 Technology Stack.....	9
4.2 Source Organization.....	9
4.3 Player and Avatar Control.....	10
4.4 Gameplay Evidence	10
4.5 Platform Services	11
CHAPTER 5 VERIFICATION AND RESULTS	12

5.1 Verification Method.....	12
5.2 Structural Results	12
5.3 Play Mode Results	12
5.4 Level Content Coverage.....	13
5.5 Limitations and Risks.....	13
5.6 Recommended Next Steps	14
CHAPTER 6 CONCLUSION.....	15
CHAPTER 7 REFERENCES	16
CHAPTER A APPENDIX: CODEBASE TRACEABILITY	17
A.1 Runtime Inventory	17
A.2 Project Inventory	17
CHAPTER B APPENDIX: LEVEL-ACTION COVERAGE	18

LIST OF FIGURES

No.	Figure
3.1	Plummet runtime architecture
3.2	Player finite-state model
3.3	Level lifecycle and event flow
3.4	Data-driven level action model
4.1	Level 1 Play Mode capture
4.2	Representative hazard and platform composition

ABBREVIATIONS

Abbreviation	Meaning
API	Application Programming Interface
DI	Dependency Injection
FSM	Finite-State Machine
FPS	Frames Per Second
SDK	Software Development Kit
SFX	Sound Effects
UI	User Interface
URP	Universal Render Pipeline
UX	User Experience
UPM	Unity Package Manager

CHAPTER 1

INTRODUCTION

1.1 Background

Puzzle-platformers combine physical navigation with systems that require the player to infer rules, predict consequences, and execute precise actions. On a mobile device, this creates a demanding interaction problem: the game must remain visually legible, respond quickly to touch input, and communicate failure without interrupting the rapid retry rhythm that makes short challenge levels engaging.

Plummet addresses this problem through a compact 2D pixel-art presentation and a modular gameplay architecture. The player moves between an entrance and an exit while avoiding spikes, saws, fatal falls, and unstable geometry. Some levels deliberately transform the interaction model by inverting controls, changing gravity or jump allowances, transferring control to a puppet, or executing a timed sequence of environmental actions.

1.2 Problem Statement

A conventional platformer can hard-code each challenge directly into a level script, but that approach becomes difficult to extend and verify as the number of levels grows. Plummet therefore requires a system that can express unusual level behavior from reusable parts, preserve consistent player rules, support fast death-and-retry loops, and integrate mobile platform services without coupling them to gameplay code.

1.3 Objectives

- Design a responsive movement and jump model suitable for desktop testing and touch-screen play.
- Represent player behavior with explicit states so movement, death, locking, and scripted control remain predictable.
- Create a reusable level-action framework for movement, waits, object disabling, physics-rule changes, control switching, camera shake, and loops.
- Build a persistent level-selection and progression system for a catalog of short puzzle stages.
- Separate gameplay from advertising, analytics, recording, audio, saving, and update services through interfaces.
- Validate the submitted project through structural inspection and direct Play Mode observation.

1.4 Scope

The project scope includes the Android game client, bootstrap/menu/gameplay scenes, player and puppet control, level catalog and instantiation, environmental actions, audio settings, local progress, analytics events, mobile advertisements, session recording integration, and in-app update support. Server-hosted gameplay, online accounts, multiplayer, cloud saves, and a remote level editor are outside the present scope.

CHAPTER 2

REQUIREMENTS ANALYSIS AND FEASIBILITY

2.1 Functional Requirements

ID	Requirement	Verified implementation evidence
FR-01	Start and navigate the game	Bootstrap loads the configured start scene; menu exposes Play, Music, and SFX controls.
FR-02	Select unlocked levels	LevelSelection uses catalog entries and saved highest-unlocked progress.
FR-03	Move and jump	Input System actions feed the player state machine and on-screen controls.
FR-04	Die and retry quickly	Hazards publish death; the runtime reloads the same level after the death sequence.
FR-05	Complete and advance	Exit door locks the avatar, animates closure, persists progress, and requests the next level.
FR-06	Alter level rules	Serializable actions can invert input, adjust gravity/jump rules, and switch the controlled avatar.
FR-07	Persist preferences	PlayerPrefs stores music, sound-effect, and level-progress values.
FR-08	Provide mobile services	Ads, analytics, Smartlook, and in-app updates are wrapped behind runtime services.

2.2 Non-Functional Requirements

Quality	Requirement and design response
Responsiveness	Input is read through a single input contract; movement and physics adjustments are applied in dedicated player states.
Maintainability	Interfaces, dependency injection, state classes, signal-driven orchestration, and catalog assets limit direct scene dependencies.
Extensibility	New level behavior can be assembled from serializable actions and sequences without rewriting the central level runtime.
Visual clarity	Pixel-art scenes use a 2D rendering stack, constrained framing, and a border overlay for aspect-ratio differences.
Resilience	Analytics queues events until initialization; service implementations isolate platform-specific dependencies.
Usability	Large on-screen directional and jump controls support the primary mobile interaction path.

2.3 Feasibility

Technical feasibility is supported by Unity 6000.3.18f1, the Universal Render Pipeline, the Input System, and established packages for asynchronous work, dependency injection, tweening, and platform services. The project builds around a small number of scenes and instantiates level prefabs from a catalog, keeping content iteration separate from scene topology.

Operational feasibility is strengthened by short self-contained levels, immediate failure feedback, saved unlock progress, and familiar touch controls. Economic feasibility is supported by the use of broadly available Unity tooling and optional advertising, while the present implementation avoids a required custom backend. Schedule feasibility is improved by the reusable level-action framework and editor utilities such as the sprite-combining tool.

2.4 Constraints and Assumptions

- The shipping target is Android; some external services require a physical device, valid credentials, or Play Store context for complete verification.
- The current catalog exposes 21 levels even though a Level 22 prefab exists in the project.
- The inspected input asset defines keyboard actions (A, D, and Space) in addition to scene-based touch controls.
- The current report verifies behavior and configuration but does not claim store-release certification or device-matrix coverage.

CHAPTER 3

SYSTEM METHODOLOGY AND DESIGN

3.1 Development Method

Plummet follows an iterative, component-oriented game-development method. Core movement and scene flow form the stable foundation; each puzzle stage composes prefabs, triggers, actions, and level rules on top of that foundation. Reusable contracts are defined first, platform implementations are injected at the composition root, and gameplay coordination occurs through typed signals. This keeps experimentation local while preserving deterministic lifecycle boundaries.

3.2 Runtime Architecture

The software can be understood as four cooperating layers. Presentation and input collect player intent and display menus, controls, camera effects, and fades. The gameplay domain owns player states, avatar behavior, hazards, doors, and level actions. Orchestration selects and loads content, transfers control, saves progress, and routes signals. Platform and data adapters communicate with PlayerPrefs and external mobile SDKs.

PLUMMET RUNTIME ARCHITECTURE

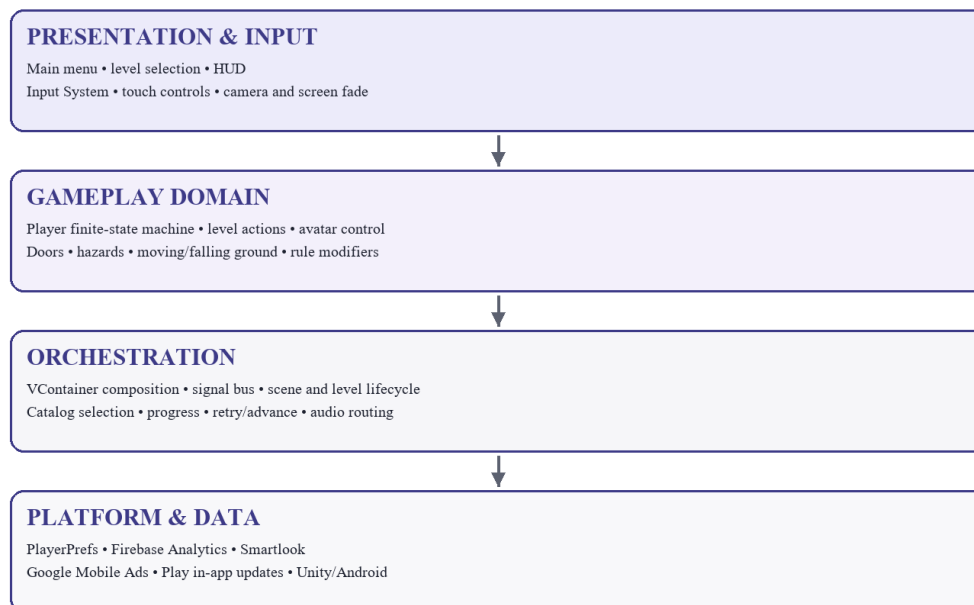


Figure 3.1: Plummet runtime architecture

3.3 Composition and Scene Lifecycle

RootLifetimeScope is the application composition root. It registers the level catalog, input,

audio library, settings, signal bus, scene loader, level selection, persistence, advertisements, analytics, Smartlook, in-app updates, and bootstrap entry point. Scene-specific lifetime scopes extend that root for menu and gameplay dependencies. Bootstrap, MainMenu, and Gameplay are the three enabled build scenes; additive loading and a screen fader hide transitions and allow the root services to remain available.

3.4 Player Finite-State Machine

The player controller delegates behavior to five explicit states. Idle waits for movement or jump input, Run applies horizontal movement, Jump manages airborne motion and enhanced falling, Locked prevents ordinary control during scripted events, and Dead prevents further play until the level runtime retries. Default inspected values are movement speed 6, jump force 12, and fall multiplier 1; level rules may modify the effective jump and gravity behavior.

PLAYER STATE MODEL

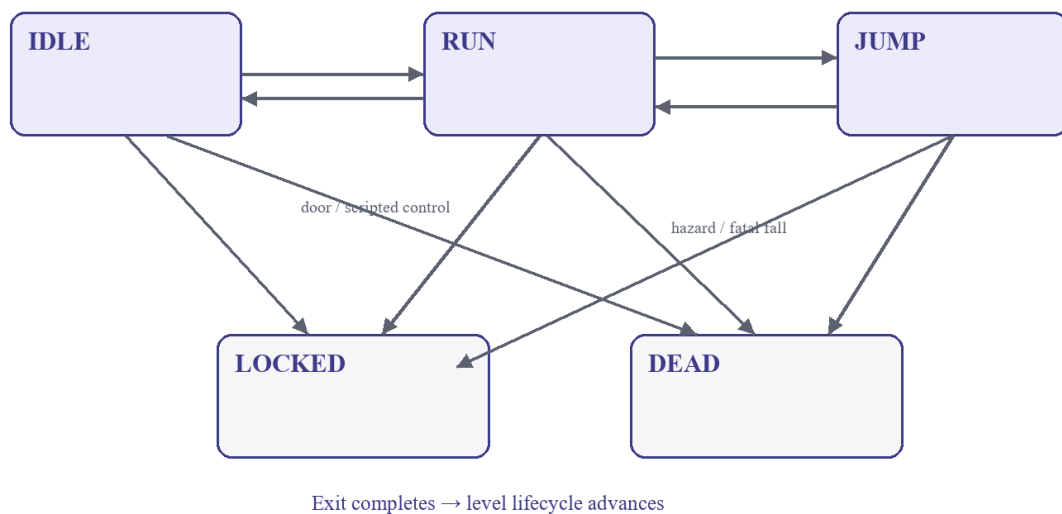
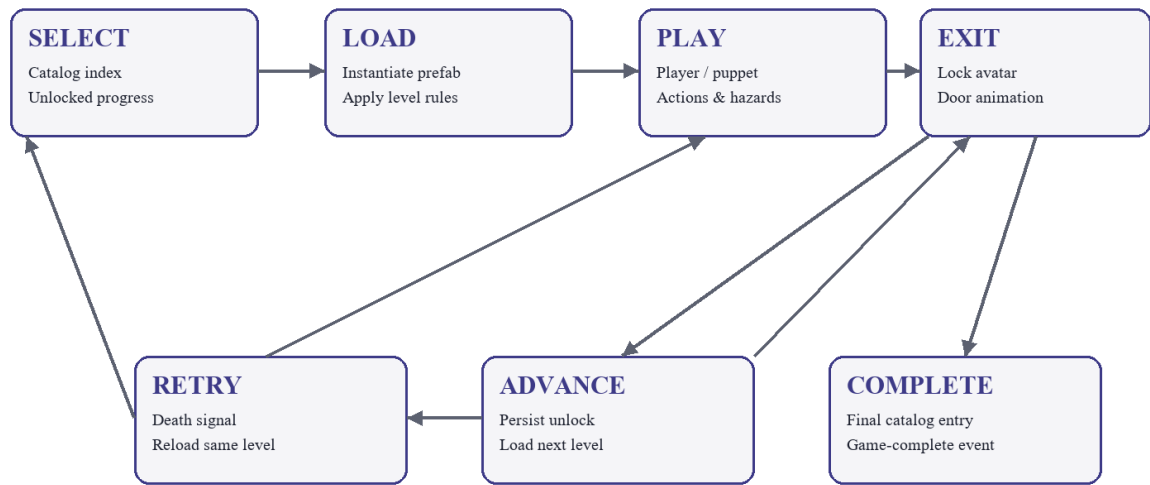


Figure 3.2: Player finite-state model

3.5 Level Lifecycle

LevelSelection identifies the active catalog entry. The runtime instantiates that level prefab through VContainer, applies its rules, spawns or binds the avatar, and records the level-start event. A death signal increments the retry path and reloads the same selection; an exit sequence records completion, unlocks progress, and advances. Reaching the final catalog entry publishes a game-complete event. Interstitial display is attempted after a randomized interval of seven to ten deaths when an advertisement is ready.

LEVEL LIFECYCLE AND EVENT FLOW



Cross-cutting signals: level started • player died • level completed • game completed

Figure 3.3: Level lifecycle and event flow

3.6 Data-Driven Level Actions

The central extensibility mechanism is a serializable action hierarchy. Actions can run sequentially or in parallel and are commonly activated by a Player-tag trigger zone or by automatic start. Implemented operations include moving an object, returning it, waiting, disabling an object, camera shaking, setting or flipping inverted controls, changing gravity and jump parameters, enabling or disabling jumping, switching the controlled avatar, applying or restoring level rules, and looping a sequence.

DATA-DRIVEN LEVEL ACTION MODEL

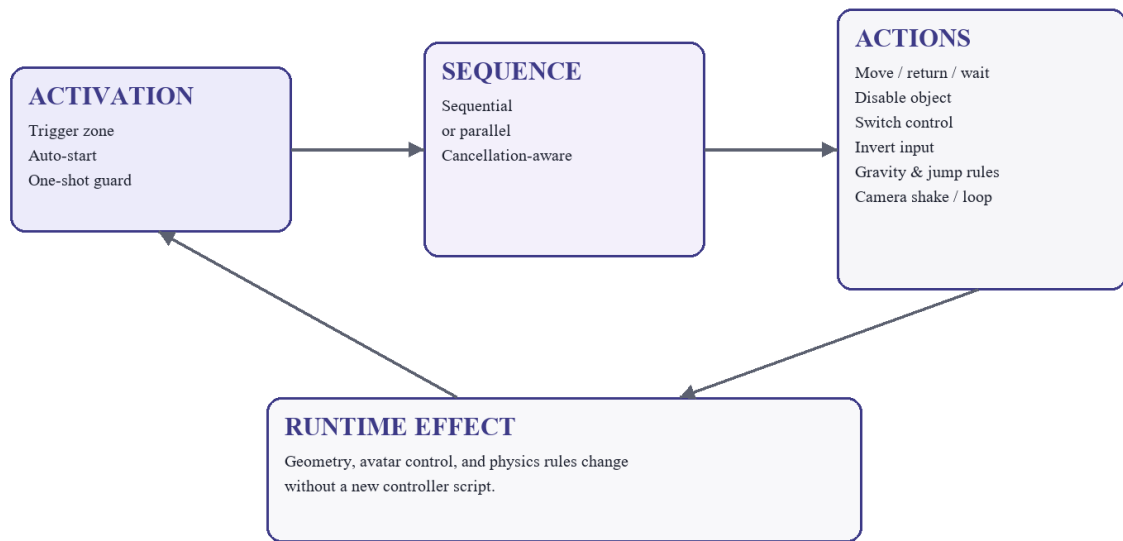


Figure 3.4: Data-driven level action model

3.7 Data and Integration Design

Concern	Primary mechanism	Boundary
Progress	Highest unlocked level in PlayerPrefs	ILevelProgress / ILevelSelection
Audio	Music and SFX settings plus AudioLibrary	IAudioService
Analytics	Queued Firebase events	IAnalyticsService
Session insight	Mobile Smartlook integration	ISmartlookService
Advertising	Banner, interstitial, rewarded units	IAdsService
Updates	Google Play in-app update flow	InAppUpdate service
Coordination	Typed publish/subscribe messages	ISignalBus

CHAPTER 4

IMPLEMENTATION

4.1 Technology Stack

Technology	Inspected version/source	Role
Unity	6000.3.18f1	Editor, engine, scenes, serialization, Android build target
URP	17.3.0	2D rendering pipeline
Input System	1.19.0	Keyboard and touch-control input actions
VContainer	Git dependency, tag 1.18.0	Dependency injection and lifetime scopes
UniTask	UPM/Git package	Allocation-conscious asynchronous workflows
PrimeTween	Local package archive	Screen, camera, and door/player tweening
Firebase Analytics	Project plugin	Gameplay event reporting
Google Mobile Ads	Project plugin	Banner, interstitial, and rewarded advertisements

4.2 Source Organization

The inspected authored code contains 115 C# files and 7,956 lines under Assets/DayZeroGames/Scripts. Of these, 106 are runtime scripts, eight are editor scripts, and one is the generated input wrapper. The code is organized by contracts, installers/lifetime scopes, player and puppet controllers, level runtime/actions, menu UI, services, audio, camera effects, and editor tooling.

Module	Representative responsibilities
Contracts	Ads, analytics, audio, input, saving, scenes, state, avatars, progress, and signals
Bootstrap & DI	Global registrations, startup entry point, update service, scene scopes
Player	FSM, movement, jump, death, collision and hazard responses
Level Runtime	Catalog selection, prefab instantiation, spawn, retry, next-level and completion
Level Actions	Sequence execution, movement, waits, rule changes, control switching and loops
Presentation	Main menu, level buttons, on-screen controls, screen fade and camera border
Services	PlayerPrefs, Firebase, Smartlook, ads, audio, and scene loading
Editor	Sprite combiner and supporting inspectors/utilities

4.3 Player and Avatar Control

Input is exposed through an abstraction rather than queried throughout gameplay. The normal player and puppet controller consume the same rule source, allowing a level to transfer control without changing the overall lifecycle. Horizontal movement is physics-based, jumping respects maximum air jumps and jump-force multipliers, and falling can be accelerated. When an exit is reached, the avatar is locked, tweened into the door, faded, and followed by a close-door sequence before advancement.

4.4 Gameplay Evidence

The following capture was taken from the submitted project in Play Mode. It shows the pixel-art composition, entrance and exit doors, player avatar, layered platforms, spike hazards, and large mobile controls. The framing also demonstrates the intended high-contrast separation between safe surfaces, hazards, and the dark background.

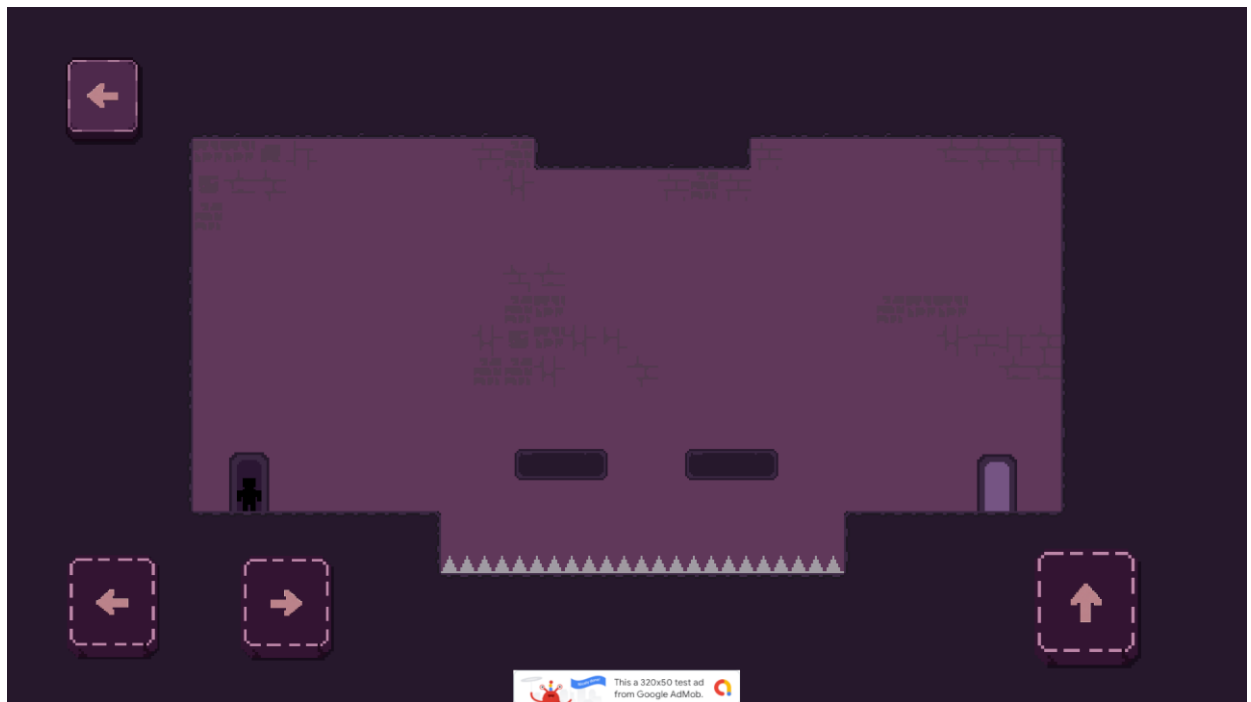


Figure 4.1: Level 1 Play Mode capture

Levels combine direct objects with nested prefabs. Across the level set, the authored action sequences use moving and returning platforms, timed waits, object removal, changes to gravity and control direction, puppet control, parallel execution, and repeated loops. Later prefabs increase the density of moving or falling geometry and hazards while reusing the same runtime contracts.

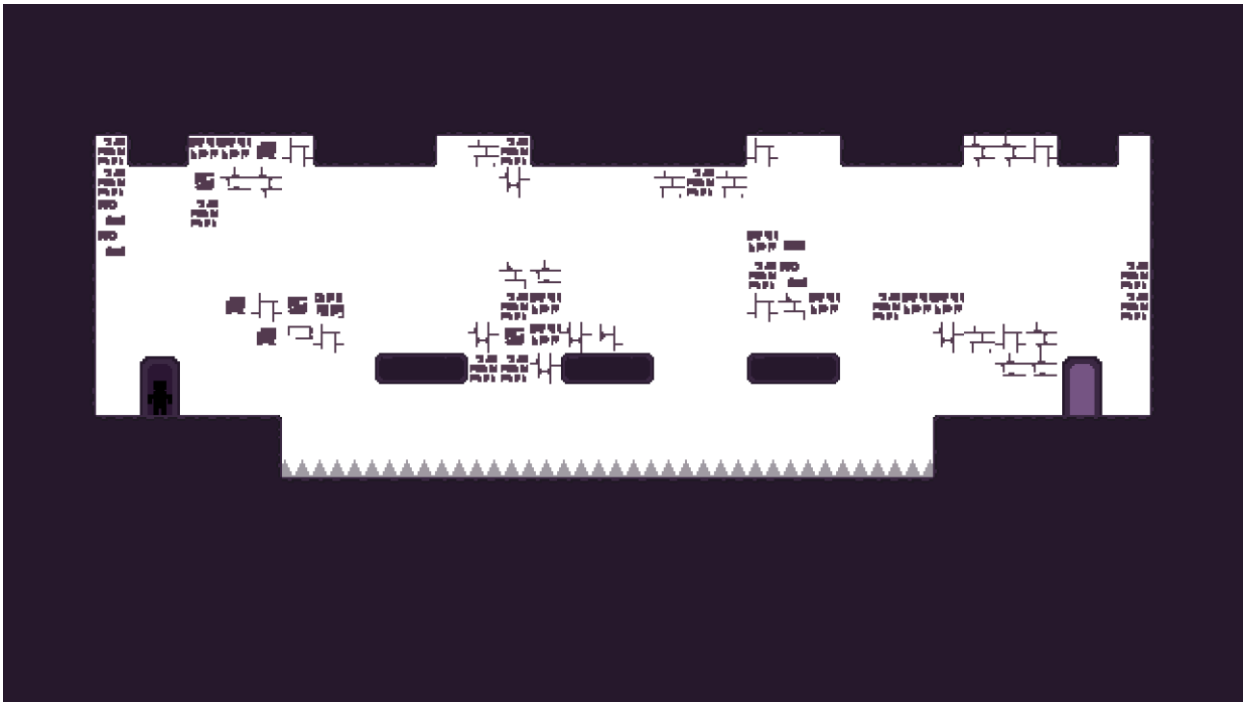


Figure 4.2: Representative hazard and platform composition

4.5 Platform Services

Platform dependencies are not called directly from the player or level code. The analytics service queues gameplay events until Firebase becomes ready and emits `player_died`, `level_started`, `level_completed`, and `game_completed` data. The advertising service manages banner, interstitial, and rewarded formats. Smartlook is restricted to supported mobile builds, and the Play in-app update integration handles Android distribution updates. This separation allows gameplay to remain testable in the Editor when external services are unavailable.

CHAPTER 5

VERIFICATION AND RESULTS

5.1 Verification Method

Verification combined static and dynamic evidence. Static inspection covered every authored C# script, build settings, tags and layers, the package manifest, ProjectSettings, LevelCatalog.asset, scene hierarchy, and all 22 level prefabs. Dynamic inspection entered Play Mode, observed the MainMenu and Gameplay scenes, loaded representative level prefabs through the runtime, captured the visible game view, and examined the Unity Console.

5.2 Structural Results

Item	Observed result	Status
Authored C# source	115 files / 7,956 lines inspected	Verified
Build scenes	Bootstrap, MainMenu, Gameplay	Verified
Level catalog	21 ordered entries, Level 1 through Level 21	Verified
Additional content	Level 22 prefab exists outside the catalog	Attention
Prefabs	57 total; 22 top-level level prefabs	Verified
Rendering/input	URP 17.3.0 and Input System 1.19.0	Verified
Android metadata	Product Plummet, company DayZero Games, bundle version 1.6/code 7	Verified

5.3 Play Mode Results

Check	Evidence	Outcome
Startup	Bootstrap transitioned to the configured menu scene	Pass
Gameplay load	Representative level prefabs instantiated and rendered	Pass
Player presentation	Player, doors, platforms, hazards, and mobile controls visible	Pass
Console	No script exception or compilation error observed in the verification pass	Pass
External configuration	Editor automation/audio-listener timing and Firebase database-URL warnings appeared	Review

The observed warnings did not prevent representative levels from loading.

5.4 Level Content Coverage

Levels	Observed composition emphasis
1	Control inversion, reduced gravity, puppet switching, moving/returning objects, spikes and falling ground
2–3	Static hazard introduction followed by triggered movement, waits, and object disabling
4–6	Dense parallel fall/move sequences, fake or falling geometry, and moving platforms
7–10	Nested/static challenge patterns with spikes, repeated movement, returns, waits, and timed disabling
11–16	Larger trigger networks, parallel sequences, moving platforms, spikes, and falling-ground combinations
17–19	Primarily nested prefab composition with falling-ground and spike challenges
20–21	Multiple object disables and movement sequences; Level 21 uses several parallel groups
22	Prefab includes a saw hazard but is not referenced by the runtime catalog

5.5 Limitations and Risks

- No authored automated test suite was identified; current assurance depends primarily on structural review and Play Mode testing.
- LevelCatalog and level-prefab inventory can diverge, as demonstrated by Level 22; an editor validation rule would prevent accidental omission.
- Advertising, analytics, recording, and update behavior require device builds and valid production credentials for end-to-end verification.
- The current local save uses PlayerPrefs and does not provide cloud synchronization, account portability, or tamper resistance.
- A broader device and aspect-ratio matrix is required before release-quality claims about performance and touch ergonomics.

5.6 Recommended Next Steps

- Add EditMode tests for catalog integrity, progress bounds, and rule composition, plus PlayMode tests for death/retry and exit/advance flows.
- Decide whether Level 22 should be added to the catalog or intentionally archived, and enforce the decision with editor validation.
- Run Android device tests for advertisements, Firebase events, Smartlook capture, in-app updates, audio focus, and lifecycle pause/resume.
- Establish frame-time, memory, loading-time, crash-free-session, and level-completion targets for release monitoring.

CHAPTER 6

CONCLUSION

Plummet is a modular foundation for a mobile 2D puzzle-platformer. It provides a reusable runtime for loading content, coordinating typed events, controlling multiple avatars, changing physics and input rules, saving progress, and integrating mobile services. The player state machine and data-driven action system are its strongest architectural decisions because they isolate behavior and let designers assemble unusual challenges without duplicating the central controller.

Inspection verified 115 authored C# files, three build scenes, 57 prefabs, and 21 cataloged levels. Representative Play Mode runs rendered without a script exception. The main readiness gaps are clear: resolve the uncataloged Level 22 prefab, introduce automated tests, and validate mobile service integrations on a production-like Android device and distribution track.

With those additions, Plummet can progress from a well-structured academic game project to a more reliably testable and release-ready product. Its separation of gameplay, orchestration, data, and platform concerns provides a sound base for that next phase.

CHAPTER 7

REFERENCES

- [1] Unity Technologies. Unity Manual: Input System.
<https://docs.unity3d.com/Manual/com.unity.inputsystem.html>
- [2] Unity Technologies. Universal Render Pipeline 2D documentation.
<https://docs.unity3d.com/6000.0/Manual/urp/2d-index.html>
- [3] HadashiA. VContainer: dependency injection library for Unity.
<https://github.com/hadashiA/VContainer>
- [4] Cysharp. UniTask: efficient async/await integration for Unity.
<https://github.com/Cysharp/UniTask>
- [5] Kyrylo Kuzyk. PrimeTween for Unity. <https://github.com/KyryloKuzyk/PrimeTween>
- [6] Google. Get started with Firebase for Unity. <https://firebase.google.com/docs/unity/setup>
- [7] Google. Google Mobile Ads Unity plug-in quick start.
<https://developers.google.com/admob/unity/quick-start>
- [8] Android Developers. Support in-app updates in Unity.
<https://developer.android.com/guide/playcore/in-app-updates/unity>
- [9] DayZero Games. Plummet Unity project source, assets, scenes, prefabs, package manifest, and project settings. Inspected 24 August 2026.

CHAPTER A

APPENDIX: CODEBASE TRACEABILITY

A.1 Runtime Inventory

Area	Evidence reviewed	Purpose in the implemented system
Contracts	All interface and signal definitions	Stable boundaries for services, state, avatars, saving, scenes, and events
Composition roots	Root, menu, gameplay and level scopes	Object lifetime, dependency registration, and entry-point startup
Player/Puppet	Controllers, states, rules, collisions	Movement, jumping, death, scripted locking, and alternate-avatar play
Level core	Catalog, runtime, selection, progress, exits	Content selection, instantiation, retry, advancement, and completion
Action system	All concrete actions and sequence runners	Reusable changes to geometry, time, input, physics, and control
Menu/UI	Menu and dynamic level-button scripts	Start flow, settings, and progress-aware level navigation
Platform services	Ads, analytics, recording, update, save, audio	External SDK isolation and persisted local state
Editor tooling	Sprite-combining editor utilities	Rasterized composite sprites, order preservation, trimming, and collider generation

A.2 Project Inventory

Artifact class	Count / value	Inspection note
Authored C# files	115	106 runtime, 8 editor, 1 generated input wrapper
Authored C# lines	7,956	Counted under Assets/DayZeroGames/Scripts
Scenes	3 enabled	Bootstrap, MainMenu, Gameplay
Prefabs	57	Includes 22 top-level level prefabs
Cataloged levels	21	Lvl_1 through Lvl_21 in LevelCatalog.asset
PNG assets	59	Pixel-art and interface imagery
Audio assets	4	BackgroundMusic, Death, ExitDoorReached, UIButtonPressed
Tags	9 built-in/custom	Includes Player, Obstacles, FallingGround, Finish

CHAPTER B

APPENDIX: LEVEL-ACTION COVERAGE

Level	Direct serialized action evidence
1	Disable ×1; Loop ×1; Move ×2; Return ×1; SetGravity ×1; SetInvert ×1; SwitchControl ×1
2	No direct actions; static spike challenge
3	Disable ×1; Move ×2; Return ×1; Wait ×1
4	Disable ×21; Move ×21; Wait ×10; eleven parallel sequences
5	Disable ×1; Move ×1
6	Disable ×2; Move ×6
7	No direct action payload identified in the top-level prefab
8	Disable ×2; Move ×6; Return ×3; Wait ×5
9	Disable ×1; Move ×8; Return ×2; Wait ×5
10	Disable ×1; Move ×4
11	Disable ×7; Move ×11; Wait ×9
12	Disable ×1; Move ×4; Wait ×1
13	No direct actions; falling-ground composition
14	Disable ×1; Move ×6; Return ×1; Wait ×2
15	Move ×2
16	Disable ×7; Move ×7; five parallel sequences
17–19	No direct top-level actions; nested prefabs with falling-ground/spike patterns
20	Disable ×3; Move ×3
21	Disable ×3; Move ×9; Wait ×2; four sequences, three parallel
22	No direct actions; saw hazard; prefab is not cataloged

Note: “direct” refers to serialized actions located in the inspected top-level level prefab. Nested prefab behavior may add further runtime composition.